

Conexión del Frontend con el Backend

Para integrar la aplicación React con el API REST del backend se realizaron las siguientes acciones:

1. Configuración del cliente HTTP

Se implementó un cliente HTTP utilizando Axios para realizar las peticiones al backend.

Se configuró la URL base del servidor:

`http://localhost:8080/api`

Además, se añadió un interceptor de Axios para incluir automáticamente el token JWT en cada solicitud mediante el header:

`Authorization: Bearer <token>`

Esto permite acceder a los endpoints protegidos del backend.

2. Implementación del Login

Se desarrolló la pantalla de autenticación en React.

El flujo de autenticación es:

1. El usuario ingresa username y password.
2. Se envía una petición POST al endpoint:

`/api/auth/login`

3. El backend responde con un token JWT.
4. El token se guarda en `localStorage` para reutilizarlo en futuras peticiones.

3. Manejo del token de autenticación

El token almacenado en `localStorage` es leído por el interceptor de Axios y se adjunta automáticamente a todas las peticiones hacia el backend.

De esta manera, los endpoints protegidos como:

`/api/blueprints`

`/api/blueprints/{author}`

`/api/blueprints/{author}/{name}`

pueden ser consumidos desde el frontend.

4. Implementación de servicios para consumir el API

Se creó una capa de servicios para acceder al backend:

- apiclient.js → cliente Axios configurado.
- blueprintsApi.js → funciones para consumir los endpoints del backend.
- apimock.js → servicio alternativo con datos simulados.
- blueprintsService.js → permite cambiar entre API real y mock usando una variable de entorno.

Esto permite alternar entre:

VITE_USE MOCK=true

VITE_USE MOCK=false

5. Consumo del API desde Redux

Se utilizaron Redux Toolkit y createAsyncThunk para manejar las peticiones al backend.

Las principales acciones implementadas fueron:

- fetchAuthors → obtener autores disponibles.
- fetchByAuthor → obtener los planos de un autor.
- fetchBlueprint → obtener un plano específico.

Estas acciones actualizan el estado global de la aplicación.

6. Visualización de datos en la interfaz

Los datos obtenidos del backend se muestran en la interfaz mediante:

- una tabla de blueprints
- un canvas donde se dibujan los puntos del plano seleccionado.

Cuando el usuario presiona Open, se consulta el plano en el backend y se dibuja en el canvas.

7. Manejo de seguridad

Se agregó una verificación para evitar llamadas al backend cuando no existe un token de autenticación, evitando errores 401 Unauthorized.

Con estas implementaciones se logró conectar correctamente el frontend en React con el backend REST, permitiendo autenticación, consulta de datos y visualización de los planos en la aplicación.

```
import api from "../api"

export const login = async (username, password) => {

  const response = await api.post("/auth/login", {
    username,
    password
  })

  const token = response.data.access_token

  localStorage.setItem("token", token)

  return response.data
}
```

```
package co.edu.eci.blueprints.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.*;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
public class CorsConfig {

  @Bean
  public CorsConfigurationSource corsConfigurationSource() {

    CorsConfiguration config = new CorsConfiguration();

    config.setAllowedOrigins(List.of("http://localhost:5173"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(allowCredentials: true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration(pattern: "**", config);

    return source;
  }
}
```

```
1 VITE_API_BASE_URL=http://localhost:8080/api
2 VITE_USE MOCK=false
3
```

```
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: './tests/setup.js',
    globals: true,
  },
})
```

```
import '@testing-library/jest-dom/vitest'

// ---- Canvas mock para jsdom ----
HTMLCanvasElement.prototype.getContext = () => {
  const noop = () => {}

  return {
    canvas: {},
    fillRect: noop,
    clearRect: noop,
    beginPath: noop,
    moveTo: noop,
    lineTo: noop,
    stroke: noop,
    arc: noop,
    fill: noop,
    strokeRect: noop,
    closePath: noop,
    save: noop,
    restore: noop,
    setTransform: noop,
    translate: noop,
    scale: noop,
    rotate: noop,
    transform: noop,
    drawImage: noop,
    fillText: noop,
    measureText: () => ({ width: 0 }),
    putImageData: noop,
    createLinearGradient: () => ({ addColorStop: noop }),
    createPattern: () => ({}),
    createRadialGradient: () => ({ addColorStop: noop }),
    getImageData: () => ({}),
    getLineDash: () => [],
    setLineDash: noop,
  }
}
```